

AI WEATHERWISE

Phase 8: Project Demonstration

Demo plan, script, evidence and submission checklist

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	8 of 8 – Project Demonstration
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Purpose

This document prepares and records the final demonstration of AI WeatherWise: what to show, in which order, with which data, and how the deliverables are submitted. The demonstration proves that the backend works end to end – from user registration to an AI-generated recommendation.

2. Submission Requirements

The final submission has two parts:

#	Item	Requirement	Link / status
1	GitHub repository	Must be a public repository link. All project files and documents are uploaded. The project is maintained and submitted phase-wise.	<paste public repository link>
2	Demo video link	Recorded demonstration of the working backend.	<paste demo video link>

Phase-wise structure to upload:

1. Brainstorming & Ideation Phase
2. Requirement Analysis Phase
3. Project Design Phase
4. Project Planning Phase
5. Project Development Phase
6. Project Testing Phase
7. Project Documentation Phase
8. Project Demonstration Phase

3. Demonstration Prerequisites

Item	Check
MongoDB running and reachable	☐

Item	Check
<code>.env</code> contains PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY	☐
Dependencies installed with <code>npm install</code>	☐
Server starts with <code>npm start</code> and shows the port message	☐
Postman collection imported and base URL set to the server port	☐
Screen recorder and microphone tested	☐

4. Demo Flow and Script

Suggested total time: **6–8 minutes**.

Step	Time	Action	What to say
1	0:00	Introduce the project and architecture	Problem, solution, MVC layers, technologies used.
2	1:00	Show project structure in VS Code	Routes, controllers, middleware, models, services.
3	2:00	Run <code>npm start</code>	Server connects to MongoDB, then listens on the port.
4	3:00	Register API	New user created; 201 Created with a JWT.
5	4:00	Login API	Credentials verified; 200 OK with a token.
6	5:00	Weather API	Current metrics for Bangalore; note the fallback flag.
7	6:00	AI recommendation API	Gemini advice from the weather data.
8	7:00	Summary and future scope	Alerts, subscriptions, front end.

Step 1 – Architecture

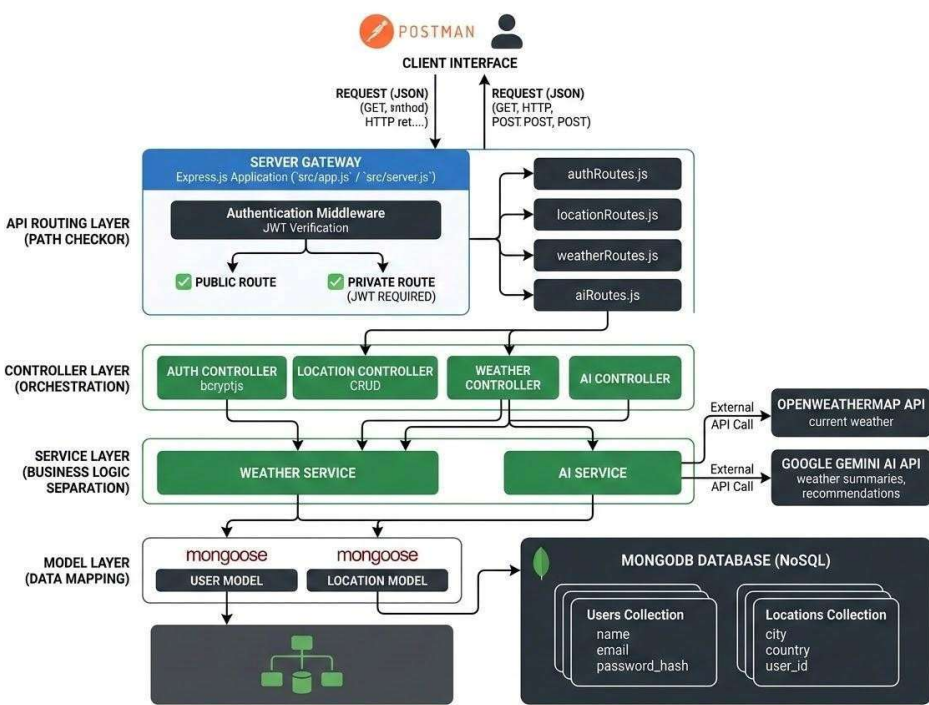


Figure 1: Architecture shown at the start of the demo

Step 2 – Project structure

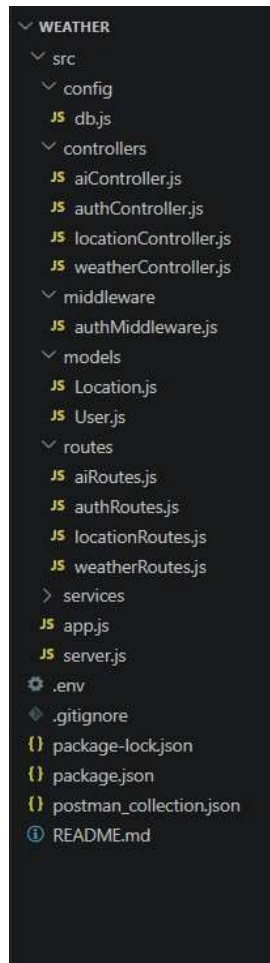


Figure 2: Backend folder structure

Step 3 – Start the server

Run `npm start`. The entry point starts Express only after the database connection succeeds.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Figure 3: Entry point (index.js) starting the server after the DB connection

Step 4 – Register

Send POST `/api/auth/register` with name, email and password.

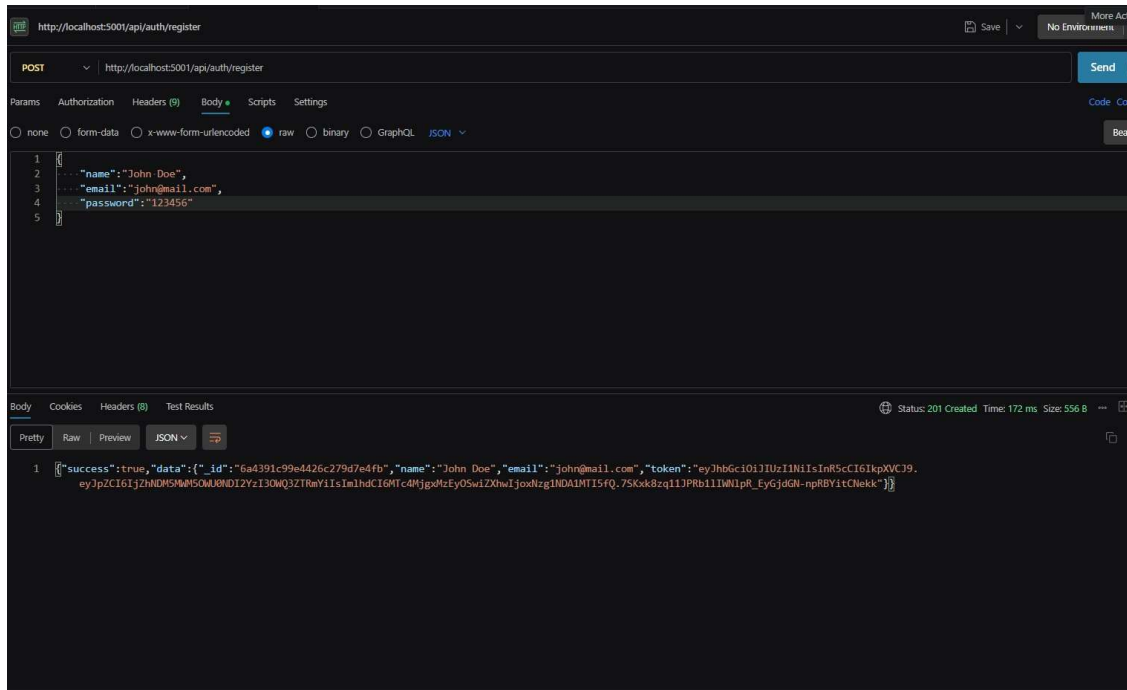


Figure 4: Register API – 201 Created

Step 5 – Login

Send POST `/api/auth/login` and show the returned token.

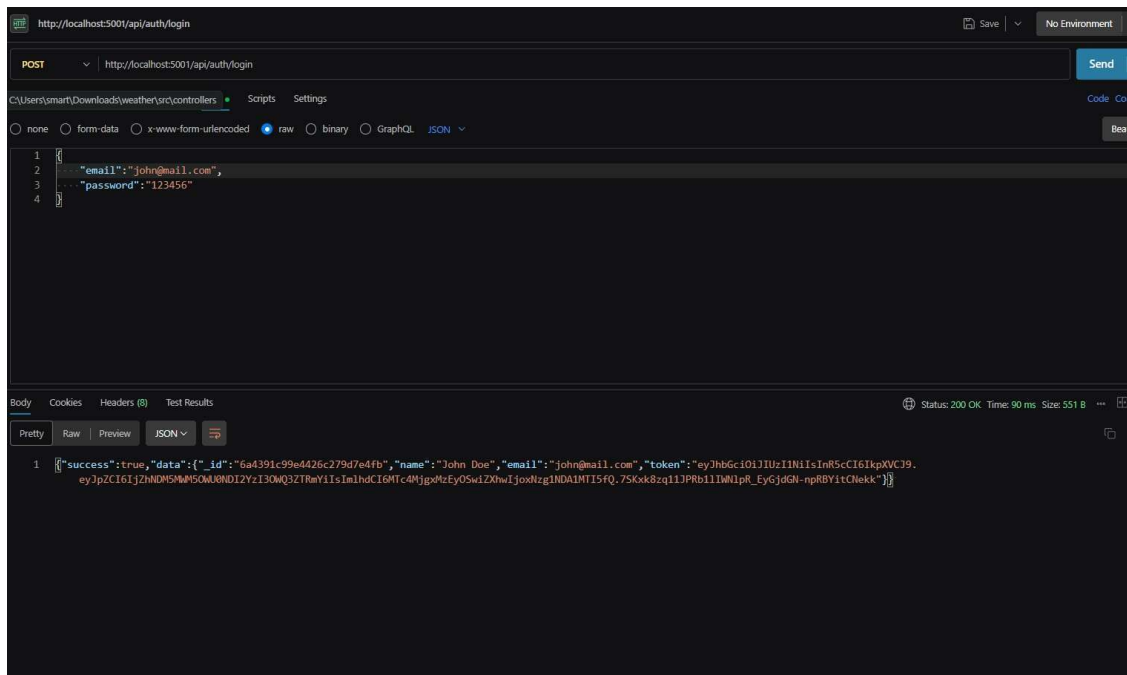


Figure 5: Login API – 200 OK

Step 6 – Weather

Send GET `/api/weather/bangalore`.

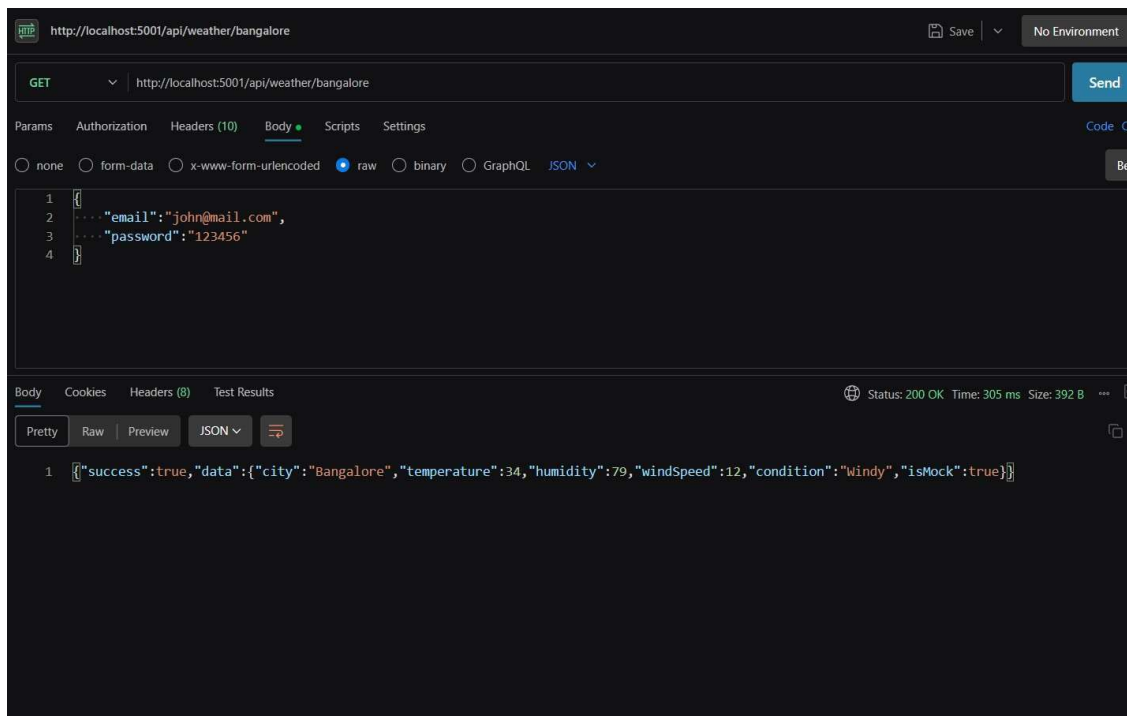


Figure 6: Weather API – 200 OK

Step 7 – AI recommendation

Send POST `/api/ai/weather-recommendation` with the weather values.

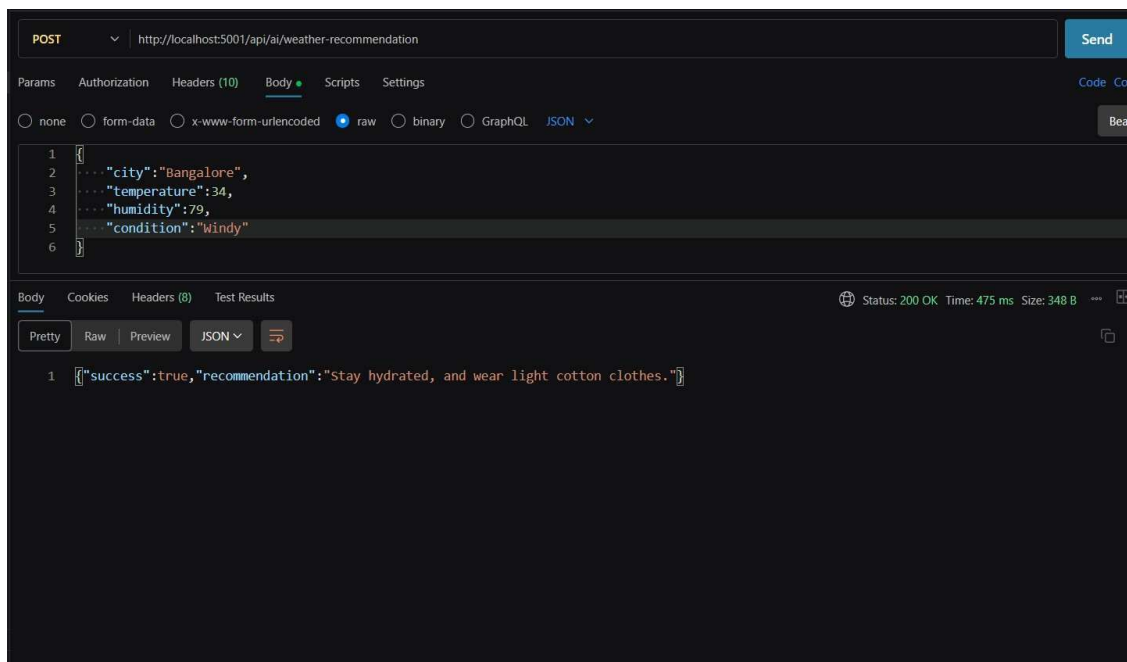


Figure 7: AI Recommendation API – 200 OK

5. Demo Data

Field	Value
Name	John Doe
E-mail	john@mail.com

Field	Value
Password	123456
City	Bangalore
Weather sample	Temperature 34, humidity 79, condition Windy

6. Video Recording Checklist

- Close unrelated windows and notifications; use a readable font size.
- Speak clearly and follow the script order.
- Show both the request and the full response in Postman.
- Keep the video under 10 minutes and export as MP4.
- Upload the video and copy a shareable link (anyone with the link can view).

Videos referenced by the project: **API testing.mp4** (API test walkthrough) and **Backend demo.mp4** (full backend demonstration).

7. Expected Questions and Answers

Question	Answer
Why MongoDB?	Flexible documents and simple user-to-location references using Mongoose.
How is security handled?	JWT tokens, bcrypt-hashed passwords, role-based guarding and input sanitization.
What if the weather or AI key is missing?	Fallback mode returns mock data (<code>isMock: true</code>) so the API keeps working.
How does AI add value?	Gemini converts raw metrics into clear recommendations for clothing and activities.
What is next?	Front end, alerts, subscriptions, weather history and an admin dashboard.

8. Backup Plan

- If the database fails to connect, show the recorded demo video and explain the fail-fast start-up.
- If the network is unavailable, fallback mode still returns weather and AI responses.
- Keep screenshots of every API response ready in this document.

9. Final Submission Checklist

Item	Done
All eight phase documents uploaded to the public GitHub repository	☐
Source code, <code>postman_collection.json</code> and README pushed	☐
<code>.env</code> excluded from the repository	☐

Item	Done
Repository link is public and opens without login	☐
Demo video uploaded and link tested	☐
Links pasted into the submission form	☐

Conclusion: the demonstration shows a secure, AI-enabled weather backend that turns raw weather data into actionable advice.